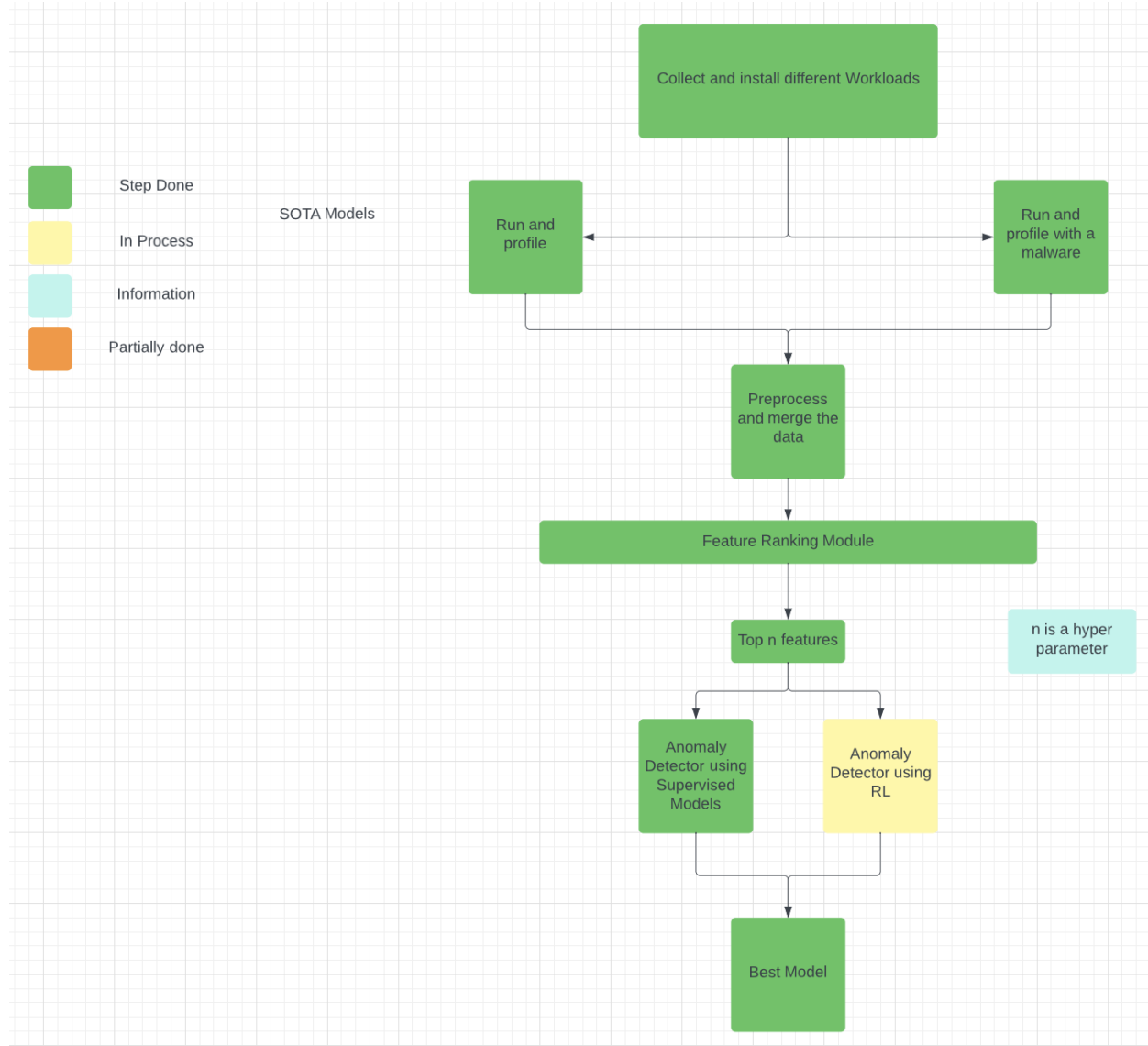


HPC BOOST

Varun Parashar(2020482), Aaditya Gupta(2020552), Samriddh Singh(2020466)

Workflow



1. Collecting and installing different workloads:

- We collected binaries for various workloads, including different and well-known applications that are readily available or have associated research papers. The range of binaries encompasses systems software (such as operating systems, network packet filters, and compilers), scientific workloads (including simulation software for weather modeling and fluid simulations), data-center workloads (representing various data structures used in data centers), and user space applications (including gaming workloads, data processing workloads, and web browsing workloads).

- Some examples of these binaries include mergesort, quicksort, Fibonacci, different machine learning algorithms (like CNN) applied to various datasets (such as CIFAR_10), Matrix Multiplication, STREAM Benchmark, UnixBench (Modified), and more.

Verify Benchmark Quality

Review Literature

- Check reputable research papers and publications to ensure that the selected benchmarks have been widely used in the academic community.

Community Adoption

- Assess the adoption of benchmarks within the research and industry communities. Benchmarks with broad usage are more likely to provide reliable and comparable results.

2. Run and profile / Collection of data (Benign)

- Utilized a Daemon Profiler Approach

A daemon is a background process that runs independently of the user. In this context, a daemon profiler refers to a program designed to monitor and collect data on the performance of a system.

The use of a daemon profiler allows for continuous and automated monitoring of system activities, providing insights into various parameters without manual intervention.

- Launched a Daemon Process with the Test Case Binary

The "test case binary" refers to the executable file of the program or application being analyzed.

We initiated a daemon process specifically tailored to the test case binary. This involves starting a background process that continuously monitors and collects data related to the execution of the test case.

- Developed a Script to Read 400+ HPC Counters on an Intel Machine

Hardware Performance Counters (HPC) are specialized registers in a computer's central processing unit (CPU) designed to store information about hardware-level events.

We created a bash script using **likwid-perfctr**, to interact with and read data from more than 400 HPC counters on an Intel machine. This script facilitates the systematic extraction of performance data.

- Using Kernel Replay for Data Collection

Kernel replay involves replaying or reproducing the behavior of the kernel (core of the operating system) to gather information.

Due to limitations in the number of Performance Monitoring Counters (PMC) events that can be read simultaneously (only 4 in this case), kernel replay is used to capture a comprehensive set of HPC data by cycling through available events.

- **Benign Data Obtained for the Test Case**

The collected data pertains to the normal or benign execution of the test case binary.

This baseline data represents the performance characteristics of the system when the test case is running under typical conditions, without the influence of external factors such as malware.

3. Run and profile with a malware / Collection of data (Malicious)

- **Execution of Malware**

Malware refers to malicious software designed to harm or exploit a computer system. In this step, the team intentionally ran various programs to mimic malwares such as mergesort and matrix multiplication (for proof of concept).

- **Profiling Process**

Profiling involves collecting detailed information about the system's behavior, resource usage, and performance during the execution of a program. We utilized the same daemon profiler to profile the system while malwares were running.

- **Collected HPC Data for the Same Test Case**

The test case mentioned here is the same as the one used in the benign scenario, providing a baseline for comparison.

During the execution of malwares, we collected HPC data, similar to the benign scenario. The difference lies in the fact that malwares are now active and potentially consuming system resources.

- **Various Malwares Running in Parallel on Different Cores**

Malwares were executed concurrently, running simultaneously on different CPU cores. Parallel execution is a common strategy for malware to maximize its impact and avoid detection.

Distribution Across Cores: Running on different cores allows malwares to exploit multiple processing units, potentially enhancing their ability to consume computational resources and evade detection mechanisms.

- **Introduced Compute Resource Consumption by Malwares**

Malwares often engage in activities that consume significant computational resources, such as CPU cycles and memory. This could include tasks like

cryptocurrency mining, distributed denial-of-service (DDoS) attacks, or other resource-intensive operations.

The intentional introduction of malwares alters the HPC data collected during profiling. The changes in resource consumption patterns, CPU utilization, and other performance metrics reflect the presence and activities of malwares on the system.

- **Resulting in Altered HPC Data**

The HPC data collected during the malicious scenario will differ from the benign scenario due to the presence of malwares.

By comparing the benign and malicious HPC data, we can identify anomalies, deviations, or patterns indicative of malicious activity. This analysis contributes to the development of detection mechanisms for identifying malware-induced performance changes.

4. Data Preprocessing and Merging

Experiment Setup 1

Objective:

Create malicious copies from benign files by introducing Additive Gaussian and Poisson noise to the Hardware Performance Counters (HPC) events.

Procedure:

Malicious Copy Generation:

For each benign file or dataset representing HPC events, create malicious copies by adding Additive Gaussian and Poisson noise.

The intent is to simulate the impact of malware on HPC events by introducing random variations that mimic the behavior of real-world malicious activities.

Data Merging:

Merge the original benign datasets with the newly generated malicious copies. Ensure that the structure and features of the datasets remain consistent.

Experiment Setup 2:**

Objective:

Generate malicious copies by running programs in parallel, simulating the presence of malware.

Procedure:

Parallel Program Execution:

Run multiple instances of programs in parallel to simulate the concurrent execution of malicious software.

This could involve executing the same benign program on multiple cores simultaneously or running different programs concurrently.

Data Merging:

Merge the datasets obtained from the parallel program execution with the original benign datasets.

Preprocessing for Both Experiment Setups:

Objective:

Convert continuous data into discrete chunks, divide it into blocks of 5 rows, and flatten each chunk into a one-dimensional vector.

Procedure:

Data Chunking:

Divide the merged dataset into continuous chunks, with each chunk containing five consecutive rows. This is done to capture short-term temporal patterns in the data.

Flattening:

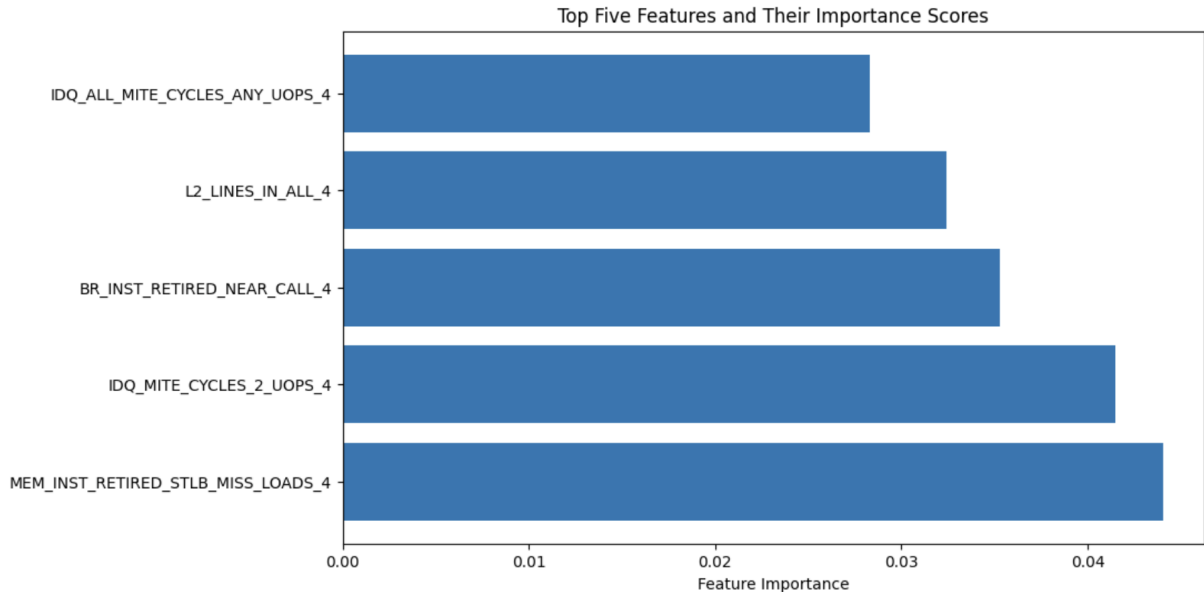
Flatten each chunk into a one-dimensional vector. This involves transforming the two-dimensional chunk into a single sequence of values.

Labeling:

Assign labels to each flattened vector based on whether it belongs to the benign set (label 0) or the malicious set (label 1).

The labeling is crucial for supervised learning, where the model learns to distinguish between benign and malicious patterns.

5. Feature Ranking



The figure displays the most important features, and it's evident that the features on core-4 have the highest significance. This core is where the malware was active. In simpler terms, our model successfully identifies the core where the malware is running.

6. Model Training

Trained classification models using the preprocessed data, and the results are promising. The models achieved very high accuracy on both the training and validation sets for both experimental setups. This suggests that the malware presence is easily detected using these methods.

7. Best Model Selection

Evaluated and selected the best-performing model.

Future Works

1. Expansion of Test Cases

- **Objective** - Enhance Dataset Robustness
- **Approach** - Introduce a diverse set of test cases to the dataset to capture a broader range of system behaviors. This may include applications with varying complexities and resource usage patterns.

2. Integration of Reinforcement Learning (RL)

- **Objective** - Improve Malware Detection Accuracy

- **Approach** - Implement Reinforcement Learning techniques to train models for malware detection. RL can adapt and learn from dynamic system behaviors, potentially improving the accuracy of detection algorithms over time.

3. Incorporation of Real-World Malwares

- **Objective** - Enhance Realism and Relevance
- **Approach** - Replace non-malicious programs (e.g., mergesort, matrix multiplication) with real-world malwares commonly encountered in the wild. This adjustment ensures that the dataset reflects scenarios closer to actual cybersecurity threats.

4. Utilization of Multi-Core Applications with Multi-Core Malwares

- **Objective** - Simulate Advanced Malware Scenarios
- **Approach** - Choose applications that utilize multiple cores for execution and introduce malwares designed to impact multiple cores simultaneously. This will provide a more realistic representation of sophisticated malware attacks on high-performance computing systems.